

SIMPLE PROCESSOR IN VHDL

Lab6

G. Dharma Teja

B20201@students.iitmandi.ac.in

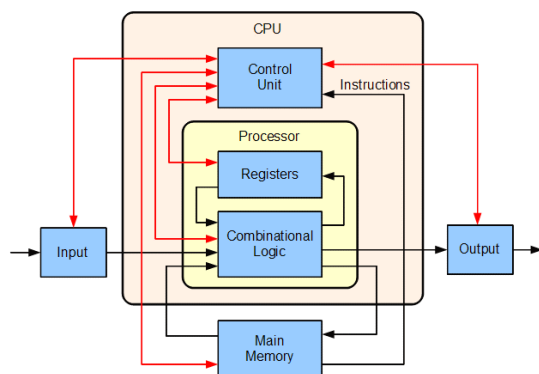
Aim

The main objective of this lab is to understand and implement a simple processor in VHDL with the following functionalities and instructions

Operation	Function
mv Rx, Ry	$R_x \leftarrow [R_y]$
mvi Rx, #D	$R_x \leftarrow D$
add Rx, Ry	$R_x \leftarrow [R_x] + [R_y]$
sub Rx, Ry	$R_x \leftarrow [R_x] - [R_y]$
and Rx, Ry	$R_x \leftarrow [R_x] \wedge [R_y]$
or Rx, Ry	$R_x \leftarrow [R_x] \vee [R_y]$
xor Rx, Ry	$R_x \leftarrow [R_x] \oplus [R_y]$
not Rx	$R_x \leftarrow \neg [R_x]$

Theory

The main theory behind this working of the simple processor is as below.



We can see that there are registers the controlling unit and ALU unit in the CPU. Registers just simply store the data and output the data when the inputs are triggered. Control unit controls the state of a register whether to load data or dump the data to bus. It takes use of multiplexer to control the switching of required register among the rest that is which register to switch. ALU simply performs the operation.

Registers and control unit need the clock to trigger i.e. they are synchronous circuits, ALU

and demultiplexer doesn't require any clock (they are combinational circuits.)

For implementing this simple processor given in the lab, these are the list of components required

1. Register (16bit) – R0 to R7,A,G
2. Decoder/rev Mux (1:16 with 4 select lines)
3. Control Unit (FSM)
4. ALU (Combinational logic)

The mechanism is as below:

Register: This can store the data when the clock transition from 0 to 1 triggers and the load is high, and gives the output when the enable is high. There we can connect a not gate to load pin and connect these 2 combinedly naming the mode pin now the TT is as below

Mode	Function
1	Dump to output
0	Load from input

This mode pin is controlled by the decoder.

Also let us assign some addresses to these registers

Register	Address
R0	0000
R1	0001
R2	0010
R3	0011
R4	0100
R5	0101
R6	0110
R7	0111
DIN	1000
A	1001
G	1010
IR	1011

Decode:

The decode activates the required register and send the value from CU to Reg.

The table for register selection with the 4 select lines is as below

Address	16 bit output
0000	ZZZZZZZZZZZZZZZZ"&val"
0001	ZZZZZZZZZZZZZZZZ"&val&"Z
0010	ZZZZZZZZZZZZZZZZ"&val&"ZZ
0011	ZZZZZZZZZZZZZZZZ"&val&"ZZZ
0100	ZZZZZZZZZZZZZZZZ"&val&"ZZZZ
0101	ZZZZZZZZZZZZZZZZ"&val&"ZZZZZ
0110	ZZZZZZZZZZZZZZZZ"&val&"ZZZZZZ
0111	ZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZ
1000	ZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZZ
1001	ZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZZZ
1010	ZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZZZZ
1011	ZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZZZZZ

Here the "val" is the mode of the register i.e. which controls the register to load in or dump out the data.

Control Unit:

For the given operations we can tabulate the steps as below (Collectively for all the instructions)

State	Process
0	DIN -> BUS
1	BUS -> IR
2	DIN -> BUS
3	BUS -> Rx
4	Rx -> BUS
5	BUS -> A
6	Ry -> BUS
7	f(A, BUS) -> G
8	G -> BUS
9	BUS -> Rx

Now the instructions can be realised as below

mv – 0,1,6,9

mvi – 0,1,2,3

all the rest – 0,1,4,5,6,7,8,9

Now for every state we can perform the required operation accordingly by just giving the signal to addr and val to de-mux. Defining a simple FSM for this control unit serves the purpose.

The format of instructions are as given in the question.

"IIIXXXYYY" – Lower 9 bits

ALU:

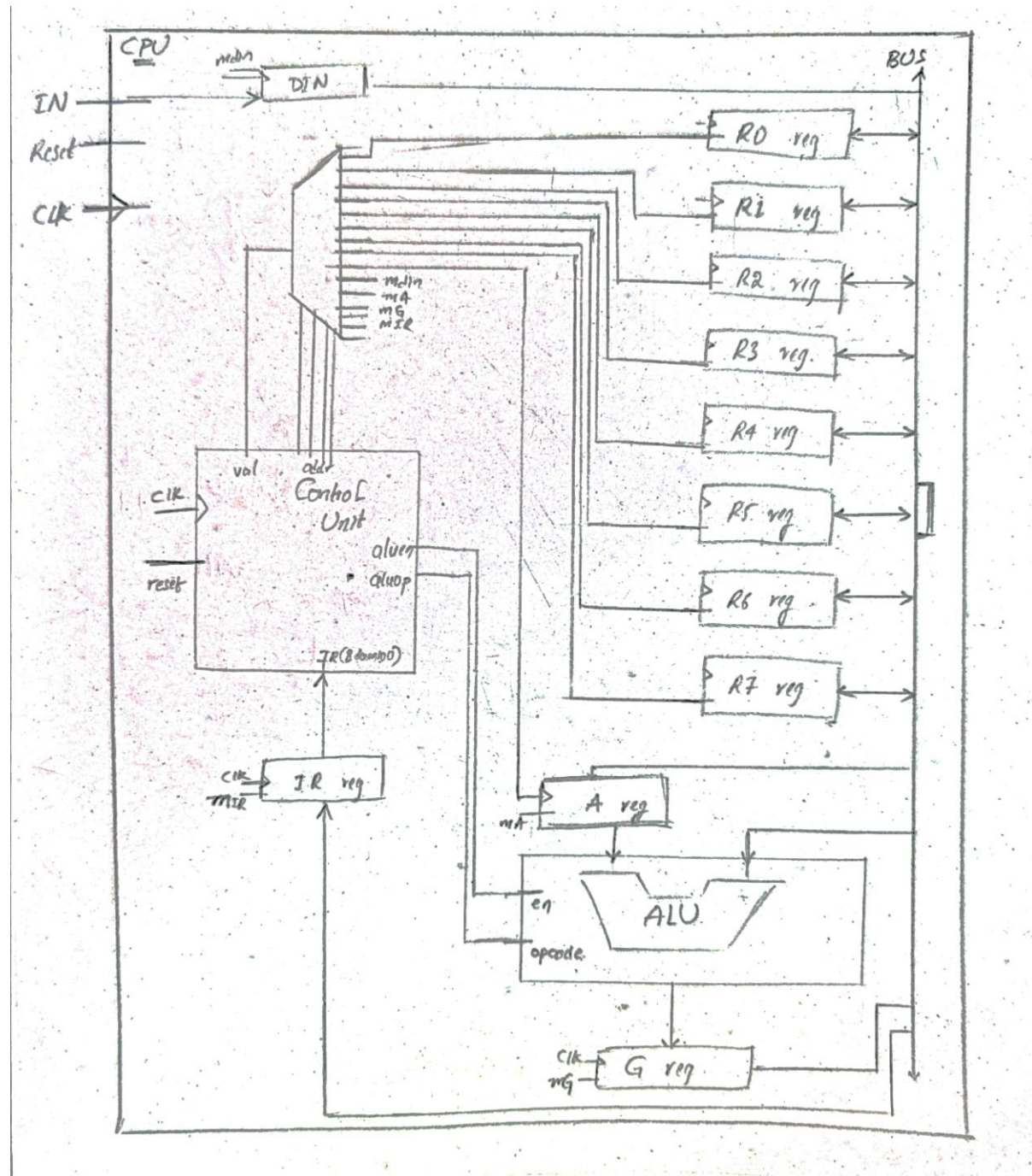
This takes the inputs, opcode. According to the opcode it performs the operation and dumps out the result.

For this project the opcodes are considered as below because they will directly correspond to the instruction at required state.

Opcode	Operation
010	Add
011	Subtract
100	AND
101	OR
110	XOR
111	Not

Circuit Diagram

The circuit / architecture can be realised as follows



Note: The schematics and implementation designs are attached at the Discussion section.

VHDL

1.The VHDL code for the register is as below

```
1  library ieee;
2  use ieee.std_logic_1164.ALL;
3  use ieee.std_logic_unsigned.all;
4
5  entity reg is
6      port(
7          din : in std_logic ;
8          clk: in std_logic ;
9          rst: in std_logic ;
10         mode : in std_logic ;
11         input : in std_logic_vector (15 downto 0);
12         output : out std_logic_vector (15 downto 0);
13         output_alu : out std_logic_vector (15 downto 0)
14     );
15 end entity;
16
17 architecture behav of reg is
18     signal t1 : std_logic ;
19     signal stored : std_logic_vector (15 downto 0) := (others=>'Z');
20 begin
21     process (clk,rst,mode)
22     begin
23         if (din = '1' and input /= "ZZZZZZZZZZZZZZZZ") then
24             stored <= input;
25         end if;
26         if rst = '1' then
27             stored <= (others=>'Z');
28         elsif rising_edge (clk) and mode='0' and din='0' then
29             stored <= input ;
30         end if;
31     end process;
32     output <= stored when mode = '1' else (others=>'Z');
33     output_alu <= stored;
34 end behav;
```

2.The VHDL code for the decoder/reverse multiplexer is as below

```
1  library ieee;
2  use ieee.std_logic_1164.ALL;
3
4  entity mux is
5      port (
6          val : in std_logic;
7          sel : in std_logic_vector (3 downto 0);
8          lines : out std_logic_vector (15 downto 0)
9      );
10 end entity;
11
```

```

12 architecture behav of mux is
13 begin
14 process (sel, val )
15 begin
16 case sel is
17 when "0000" =>
18     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val ;
19 when "0001" =>
20     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"Z";
21 when "0010" =>
22     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZ";
23 when "0011" =>
24     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZZ";
25 when "0100" =>
26     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZZZ";
27 when "0101" =>
28     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZZZZ";
29 when "0110" =>
30     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZZZZZ";
31 when "0111" =>
32     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZ";
33 when "1000" =>
34     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZZ";
35 when "1001" =>
36     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZZZ";
37 when "1010" =>
38     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZZZZ";
39 when "1011" =>
40     lines <= "ZZZZZZZZZZZZZZZZZZZZ"&val&"ZZZZZZZZZZZ";
41 when others =>
42     lines(15) <= val;
43 end case;
44 end process ;
45 end behav;

```

3.The VHDL code for the ALU is as below

```

1  library ieee;
2  use ieee.std_logic_1164.ALL;
3  use ieee.std_logic_unsigned.ALL ;
4
5  entity alu is
6  port(
7      en: in std_logic ;
8      a: in std_logic_vector (15 downto 0);
9      b: in std_logic_vector (15 downto 0);
10     op : in std_logic_vector (2 downto 0);
11     output : out std_logic_vector (15 downto 0)
12 );
13 end entity;
14
15 architecture behav of alu is
16     signal t16 : std_logic_vector (15 downto 0);
17     begin
18     process (a,b,op,en)
19     begin
20     if en = '1' then
21     case op is
22     when "010" =>
23         t16 <= a+b;
24     when "011" =>
25         t16 <= a-b;
26     when "100" =>
27         t16 <= a and b;
28     when "101" =>
29         t16 <= a or b;
30     when "110" =>
31         t16 <= a xor b;
32     when "111" =>
33         t16 <= not a;
34     when others=>
35         t16 <= (others =>'Z');
36     end case;
37     end if;
38     end process ;
39     output <= t16;
40 end behav ;

```

4.The VHDL code for the control unit FSM is as below

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity cu is
5      Port (
6          reset : in std_logic ;
7          clk : in std_logic ;
8          ir : in std_logic_vector (8 downto 0);
9          muxval: out std_logic ;
10         muxaddr : out std_logic_vector (3 downto 0);
11         aluop : out std_logic_vector (2 downto 0);
12         aluen : out std_logic
13     );
14 end entity ;
15
16 architecture Behavioral of cu is
17     signal addrx, addry : std_logic_vector (3 downto 0);
18     signal instruction : std_logic_vector (2 downto 0);
19     signal tmuxaddr : std_logic_vector (3 downto 0);
20     signal tmuxval : std_logic ;
21     signal oo : std_logic := '0';
22     type statetype is (s00,s0,s1,s2,s3,s4,s5,s6,s7,s8,s9);
23     signal st,nst : statetype := s00;
24     begin
25         addry <= "0"&ir(2 downto 0);
26         addrx <= "0"&ir(5 downto 3);
27         instruction <= ir(8 downto 6);
28         process(instruction,addrx , addry, st, tmuxaddr , tmuxval, nst )
29         begin
30             case st is
31                 when s00 =>
32                     nst <= s0;
33                 when s0 =>
34                     tmuxval <= '1';
35                     tmuxaddr <= "1000";
36                     nst <= s1;
37                 when s1 =>
38                     tmuxval <= '0';
39                     tmuxaddr <= "1011";
40                     case instruction is
41                         when "000" =>
42                             nst <= s6;
43                         when "001" =>
44                             nst <= s2;
45                         when "010" =>
46                             nst <= s4;
47                         when "011" =>
48                             nst <= s4;
49                         when "100" =>
50                             nst <= s4;
51                         when "101" =>
52                             nst <= s4;
53                         when "110" =>
54                             nst <= s4;
55                         when "111" =>
56                             nst <= s4;
57                         when others =>
58                             nst <= s0;
59                     end case;
60                 when s2 =>
61                     tmuxval <= '1';
62                     tmuxaddr <= "1000";
63                     nst <= s3;
64                 when s3 =>
65                     tmuxval <= '0';
66                     tmuxaddr <= addrx ;
67                     nst <= s0;

```

```

68         when s4 =>
69             tmuxval <= '1';
70             tmuxaddr <= addrx;
71             nst <= s5;
72         when s5 =>
73             tmuxval <= '0';
74             tmuxaddr <= "1001";
75             nst <= s6;
76         when s6 =>
77             tmuxval <= '1';
78             tmuxaddr <= addrx;
79             case instruction is
80                 when "000" =>
81                     nst <= s9;
82                 when others =>
83                     nst <= s7;
84             end case;
85         when s7 =>
86             aluen <= '1';
87             aluop <= instruction ;
88             tmuxval <= '1';
89             tmuxaddr <= "1010";
90             tmuxval <= '2';
91             nst <= s8;
92         when s8 =>
93             aluen <= '0';
94             tmuxval <= '1';
95             tmuxaddr <= "1010";
96             nst <= s9;
97         when s9 =>
98             tmuxval <= '0';
99             tmuxaddr <= addrx;
100            nst <= s0;
101        when others =>
102            nst <= s0;
103    end case;
104    end process;
105    muxaddr <= tmuxaddr;
106    clk_proc : process
107    begin
108        wait until (clk'event and clk='1');
109        if reset = '1' then
110            st <= s00;
111        elsif oo = '0' then
112            st <= nst;
113        end if;
114        oo <= not oo;
115    end process ;
116 end Behavioral;

```

5. The final VHDL code for our processing connecting all these components is as below

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity cpu is
5      Port(
6          reset : in std_logic ;
7          opclk : in std_logic ;
8          mdin : in std_logic_vector (15 downto 0);
9          outputt : out std_logic_vector(15 downto 0)
10         );
11 end entity ;
12
13 architecture structural of cpu is
14     component mux is
15     port (
16         val : in std_logic;
17         sel : in std_logic_vector (3 downto 0);
18         lines : out std_logic_vector (15 downto 0)
19     );
20 end component ;
21     component alu is
22     port(
23         en : in std_logic ;
24         a : in std_logic_vector (15 downto 0);
25         b : in std_logic_vector (15 downto 0);
26         op : in std_logic_vector (2 downto 0);
27         output : out std_logic_vector (15 downto 0)
28     );
29 end component ;
30     component cu is
31     Port (
32         reset : in std_logic ;
33         clk : in std_logic ;
34         ir : in std_logic_vector (8 downto 0);
35         muxval: out std_logic ;
36         muxaddr : out std_logic_vector (3 downto 0);
37         aluop : out std_logic_vector (2 downto 0);
38         aluen : out std_logic
39     );
40 end component ;
41     component reg is
42     port(
43         din : in std_logic ;
44         clk: in std_logic ;
45         rst: in std_logic ;
46         mode : in std_logic ;
47         input : in std_logic_vector (15 downto 0);
48         output : out std_logic_vector (15 downto 0);
49         output_alu : out std_logic_vector (15 downto 0)
50     );
51 end component ;
52 --Signals
53 signal sbusin,sbusout,sir,slines : std_logic_vector (15 downto 0);
54 signal solk : std_logic ;
55 signal srst, smuxval : std_logic ;
56 signal alua,aluo : std_logic_vector (15 downto 0);
57 signal smuxaddr : std_logic_vector (3 downto 0);
58 signal opcode : std_logic_vector (2 downto 0);
59 signal saluen : std_logic ;
60
61 begin
62 Buss: reg port map (din => '1',clk => solk,rst => srst,mode => 'U',input => sbusin, output_alu => sbusin);
63 R0: reg port map (din => '0',clk => solk,rst => srst,mode => slines(0),input => sbusin, output => sbusout);
64 R1: reg port map (din => '0',clk => solk,rst => srst,mode => slines(1),input => sbusin, output => sbusout);
65 R2: reg port map (din => '0',clk => solk,rst => srst,mode => slines(2),input => sbusin, output => sbusout);
66 R3: reg port map (din => '0',clk => solk,rst => srst,mode => slines(3),input => sbusin, output => sbusout);
67 R4: reg port map (din => '0',clk => solk,rst => srst,mode => slines(4),input => sbusin, output => sbusout);
68 R5: reg port map (din => '0',clk => solk,rst => srst,mode => slines(5),input => sbusin, output => sbusout);
69 R6: reg port map (din => '0',clk => solk,rst => srst,mode => slines(6),input => sbusin, output => sbusout);
70 R7: reg port map (din => '0',clk => solk,rst => srst,mode => slines(7),input => sbusin, output => sbusout);
71 DINN: reg port map (din => '1',clk => solk,rst => srst,mode => slines(8),input => mdin, output => sbusout);
72 A : reg port map (din => '0',clk => solk,rst => srst,mode => slines(9),input => sbusin, output_alu => alua);
73 G : reg port map (din => '1',clk => solk,rst => srst,mode => slines(10),input => aluo, output => sbusout);
74 IR : reg port map (din => '0',clk => solk,rst => srst,mode => slines(11),input => sbusin, output_alu => sir);
75 demux : mux port map (val => smuxval , sel => smuxaddr , lines => slines );
76 control : cu port map (reset => srst , clk => solk , ir => sir(8 downto 0), muxval => smuxval , muxaddr => smuxaddr , aluop => opcode, aluen => saluen);
77 aluu : alu port map (en => saluen,a => alua,b => sbusin,op => opcode,output=>aluo);
78 solk <= opclk;
79 srst <= reset ;
80 outputt <= sbusout ;
81 end structural;

```

Results of Testbench

The common testbench code is as below

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity cputb is
5  end entity;
6
7  architecture behav of cputb is
8  component cpu is
9      Port(
10         reset : in std_logic ;
11         opucclk : in std_logic ;
12         mdin : in std_logic_vector (15 downto 0);
13         outputtt : out std_logic_vector(15 downto 0)
14         );
15 end component ;
16 signal clksignal,reset : std_logic ;
17 signal input : std_logic_vector (15 downto 0);
18 constant clk_period : time := 10 ns;
19 begin
20     Processor : cpu port map (reset => reset, opucclk => clksignal, mdin => input);
21     clk_process :process
22     begin
23         clksignal <= '0';
24         wait for clk_period/2;
25         clksignal <= '1';
26         wait for clk_period/2;
27     end process;
28     stim_proc: process
29     begin
30         reset <= '0';
31         --inputs are given here.
32
33         reset <= '1';
34         input <= (others => 'U');
35         wait ;
36     end process;
37 end architecture;

```

We considered R0 and R1 registers for the task.

Let us first load 2 values into the registers and perform the different operations.

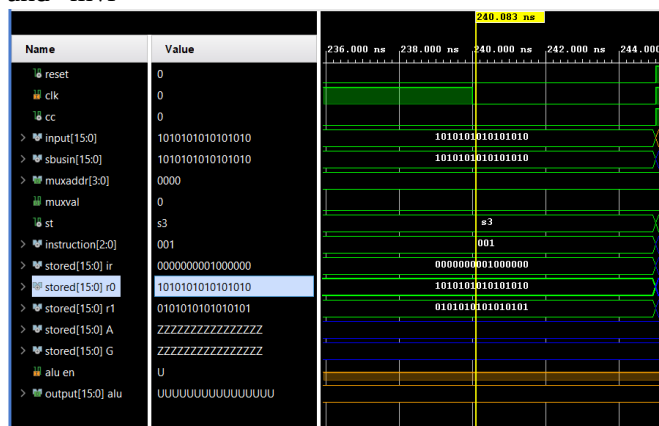
mv, mvi can be tested by loading into a register moving it to another, again loading to the same register. At this point we will be having the values loaded in 2 registers. By sending the instruction now we can test the remaining all operations.

0. Loading the data – Testing “mv” and “mvi”

```

--Input mvi
input <= "0000000001000000" ;
wait for 35ns;
input <= "0101010101010101";
wait for 30ns;
--Input mv
input <= "0000000000001000";
wait for 95ns;
--Input mvi
input <= "0000000001000000";
wait for 35ns;
input <= "1010101010101010";
wait for 30ns;

```



1. Now performing the operation on the data in R0 and R1
 - a. Add

```

--Input mvi
input <= "0000000001000000" ;
wait for 35ns;
input <= "0101010101010101";
wait for 30ns;
--Input mv
input <= "0000000000001000";
wait for 95ns;
--Input mvi
input <= "0000000001000000";
wait for 35ns;
input <= "1010101010101010";
wait for 30ns;
--adding these and storing it to R0
input <= "0000000010000001";
wait for 180ns;

```

Binary Calculator

First number

1010101010101010

Operation

add (+)

Second number

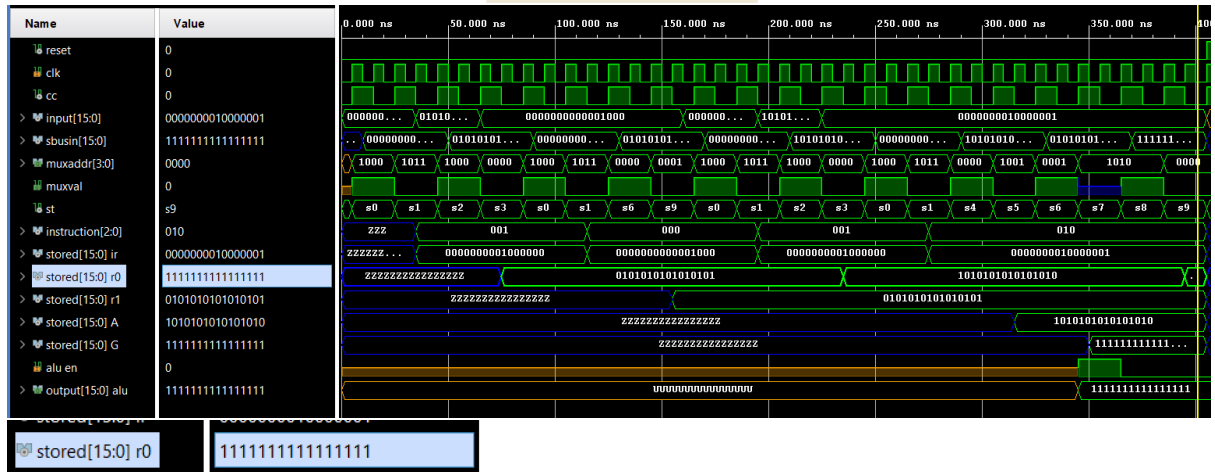
0101010101010101

= Calculate

✕ Reset

Binary result

1111111111111111



b. Subtract

```

--Input mvi
input <= "0000000001000000" ;
wait for 35ns;
input <= "0101010101010101";
wait for 30ns;
--Input mv
input <= "0000000000001000";
wait for 95ns;
--Input mvi
input <= "0000000001000000";
wait for 35ns;
input <= "1010101010101010";
wait for 30ns;
--subtracting these and storing it to R0
input <= "0000000011000001";
wait for 180ns;

```

Binary Calculator

First number

1010101010101010

Operation

sub (-)

Second number

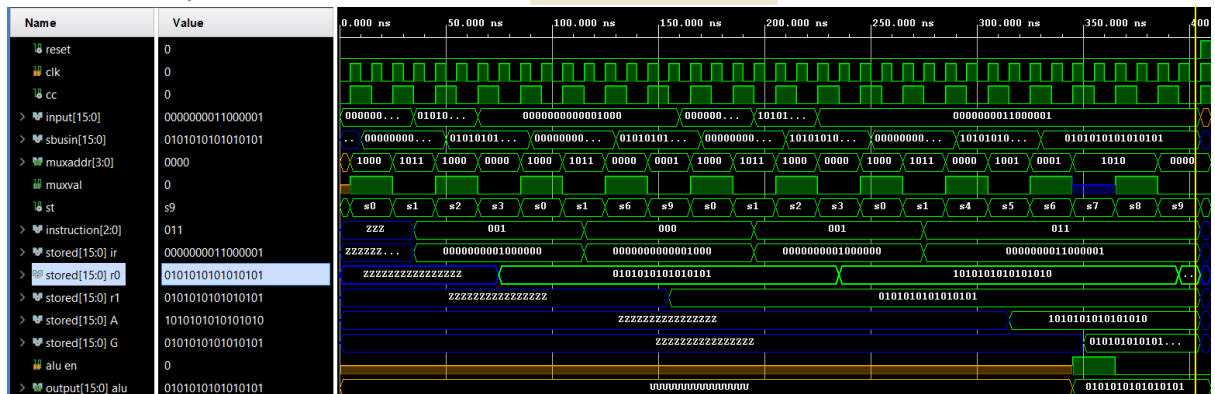
0101010101010101

= Calculate

✕ Reset

Binary result

1010101010101010



> stored[15:0] r0 0101010101010101

c. AND

```
--Input mvi
input <= "0000000001000000" ;
wait for 35ns;
input <= "0101010101010101";
wait for 30ns;
--Input mv
input <= "0000000000001000";
wait for 95ns;
--Input mvi
input <= "0000000001000000";
wait for 35ns;
input <= "1010101010101010";
wait for 30ns;
--performing "and" on these and storing it to R0
input <= "0000000100000001";
wait for 180ns;
```

Binary Calculator

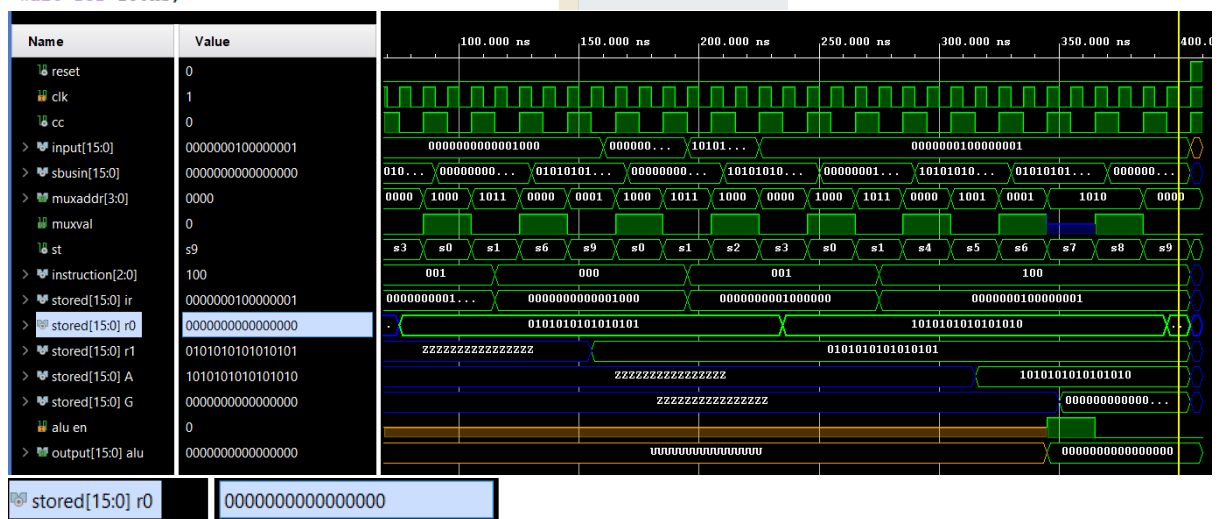
First number
1010101010101010

Operation
and (&)

Second number
0101010101010101

= Calculate × Reset

Binary result
0000000000000000



d. OR

```
--Input mvi
input <= "0000000001000000" ;
wait for 35ns;
input <= "0101010101010101";
wait for 30ns;
--Input mv
input <= "0000000000001000";
wait for 95ns;
--Input mvi
input <= "0000000001000000";
wait for 35ns;
input <= "1010101010101010";
wait for 30ns;
--performing "or" on these and storing it to R0
input <= "0000000101000001";
wait for 180ns;
```

Binary Calculator

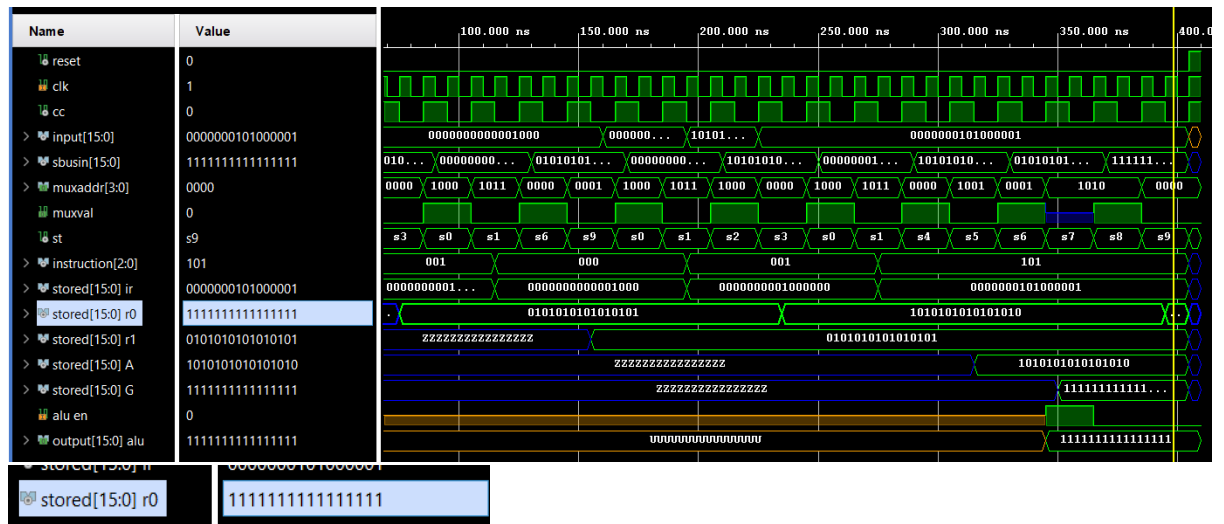
First number
1010101010101010

Operation
or (|)

Second number
0101010101010101

= Calculate × Reset

Binary result
1111111111111111



e. XOR

```
--Input mvi
input <= "0000000001000000" ;
wait for 35ns;
input <= "0101010101010101";
wait for 30ns;
--Input mv
input <= "0000000000001000";
wait for 95ns;
--Input mvi
input <= "0000000001000000";
wait for 35ns;
input <= "1010101010101010";
wait for 30ns;
--performing "xor" on these and storing it to R0
input <= "0000000110000001";
wait for 180ns;
```

Binary Calculator

First number

1010101010101010

Operation

xor (^)

Second number

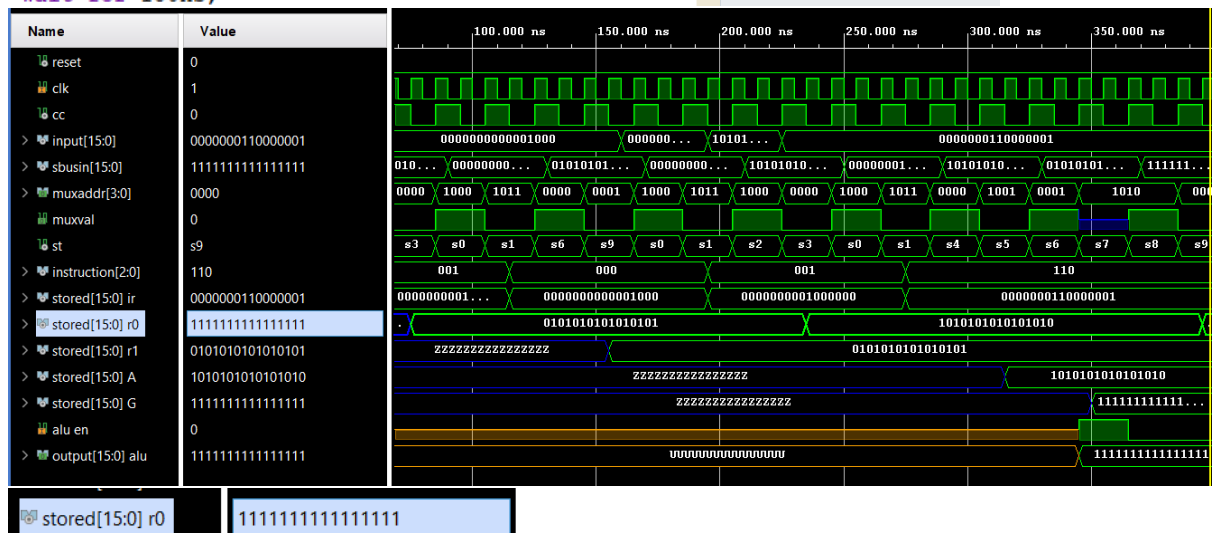
0101010101010101

= Calculate

✕ Reset

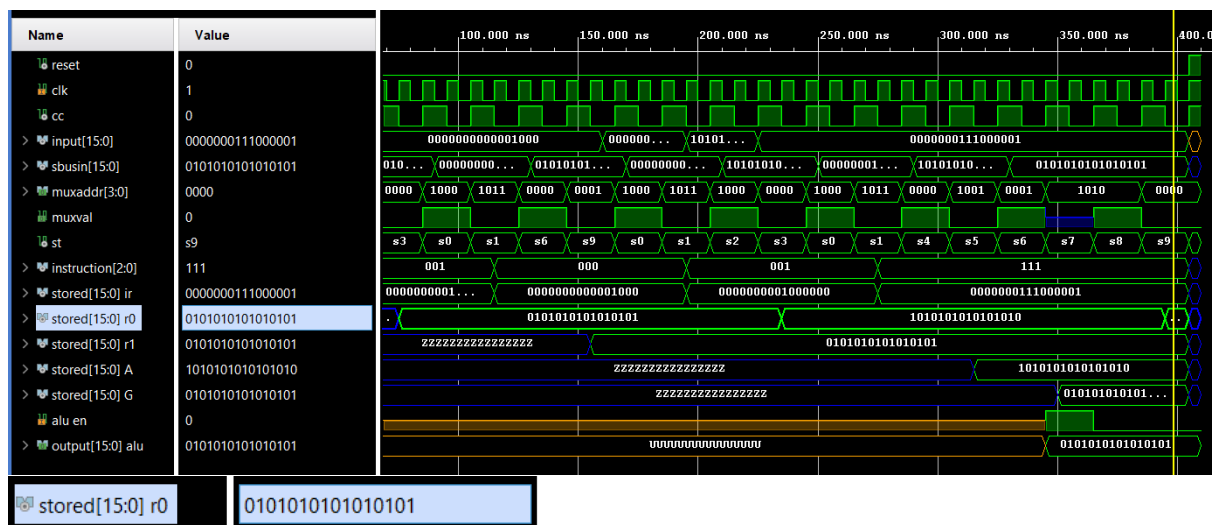
Binary result

1111111111111111



f. NOT

```
--Input mvi
input <= "0000000001000000" ;
wait for 35ns;
input <= "0101010101010101";
wait for 30ns;
--Input mv
input <= "0000000000001000";
wait for 95ns;
--Input mvi
input <= "0000000001000000";
wait for 35ns;
input <= "1010101010101010";
wait for 30ns;
--performing "not" on R0 and storing it to R0
input <= "0000000111000001";
wait for 180ns;
```



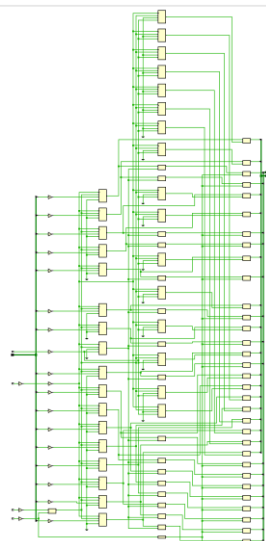
Hence we got the required results (correct results).

Discussion

The synthesis, implementation and RTL schematics are as below

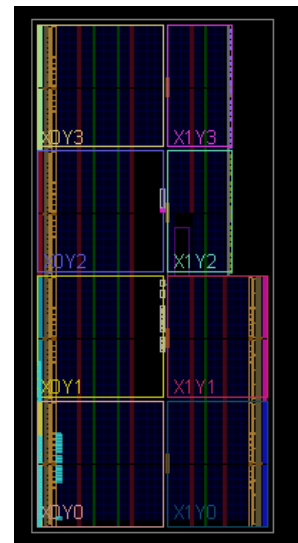
i. Registers

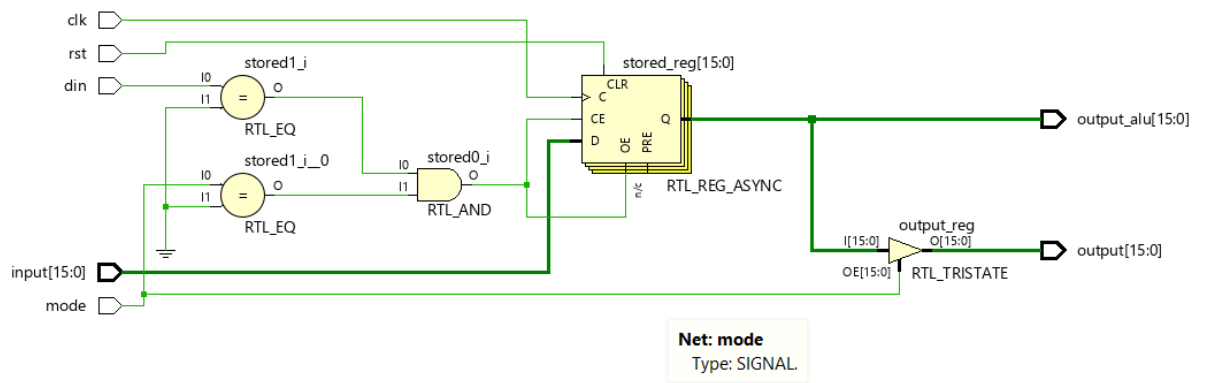
Implementation schematic



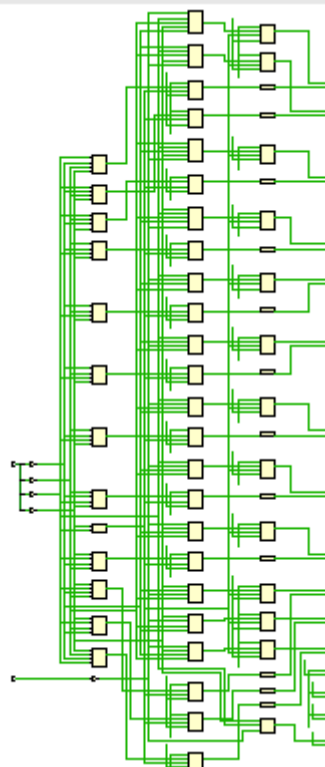
RTL schematic

Device



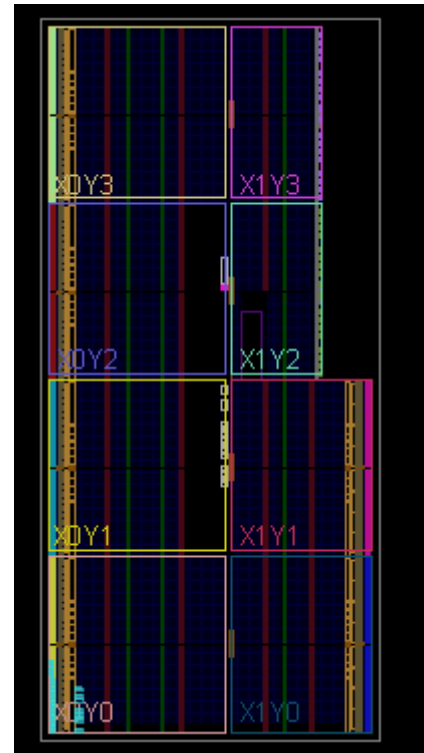


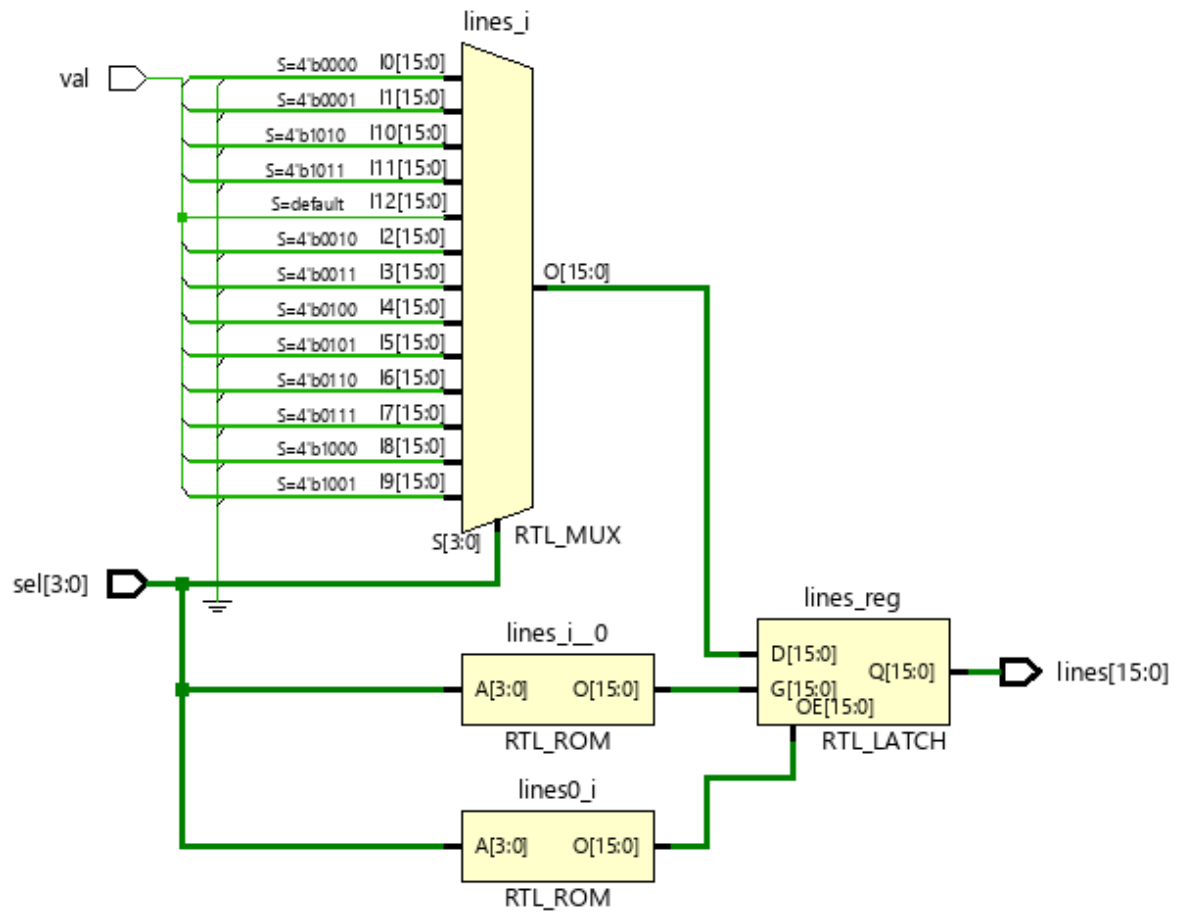
ii. Decoder/ rev Mux
Implementation Schematic



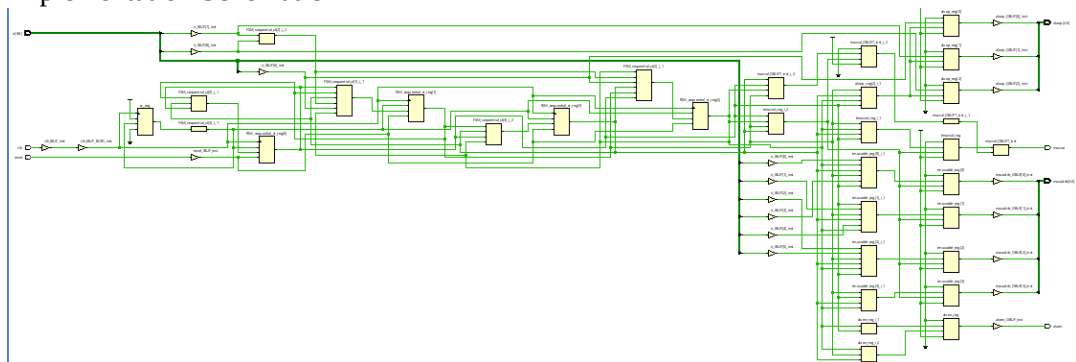
RTL schematic

Device

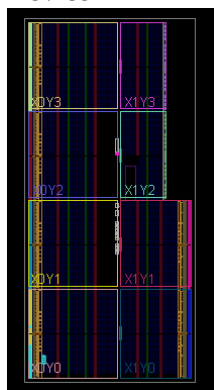




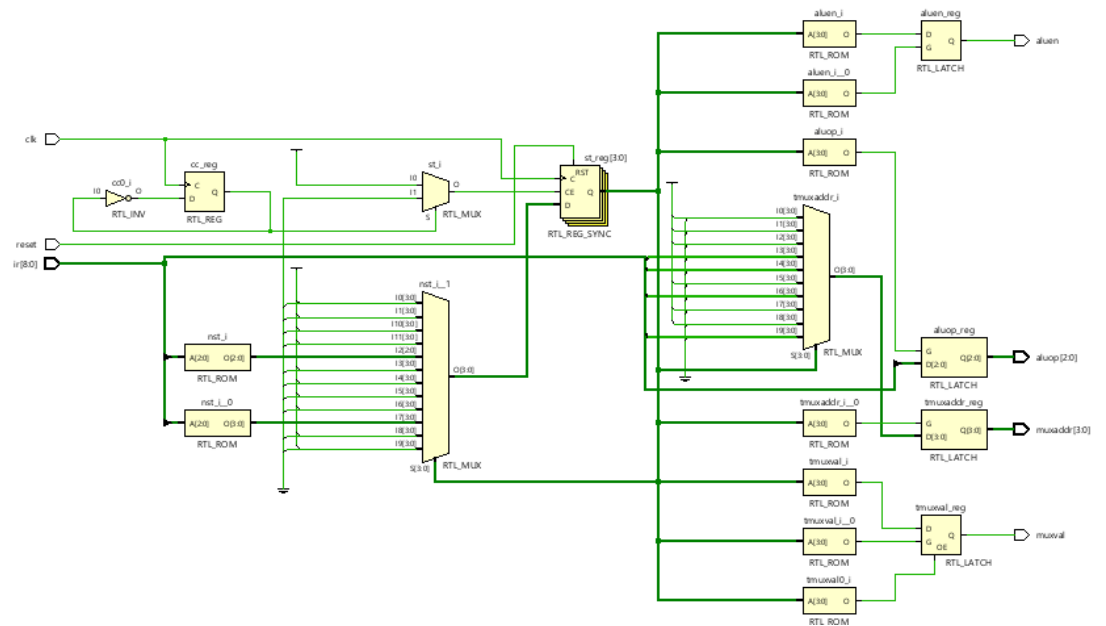
iii. Control Unit Implementation Schematic



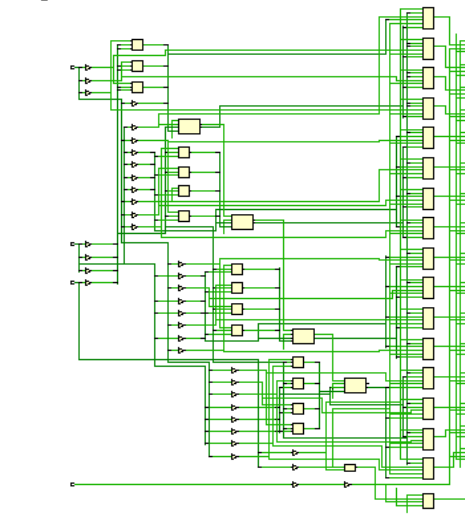
Device



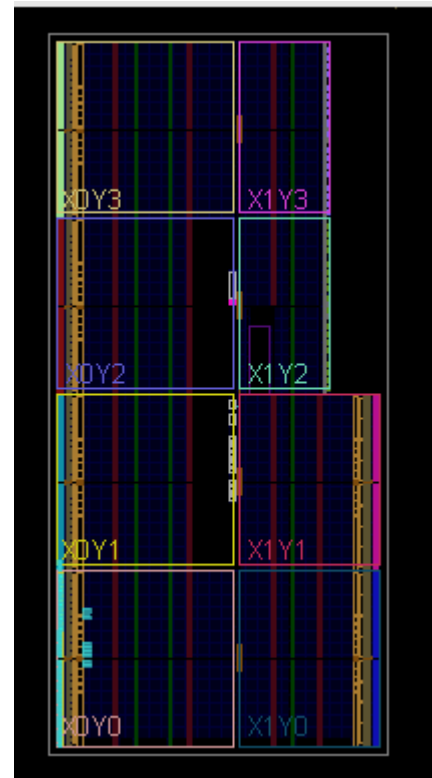
RTL schematic



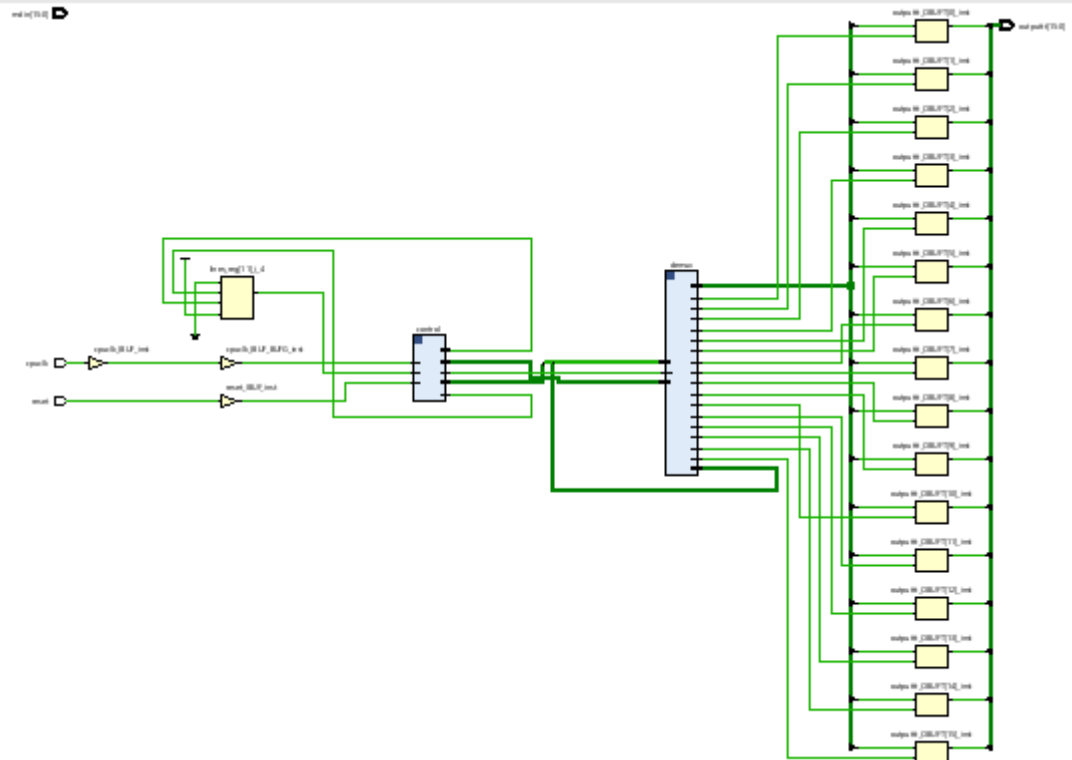
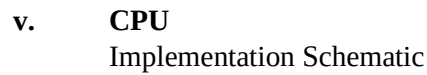
iv. ALU Implementation Schematic

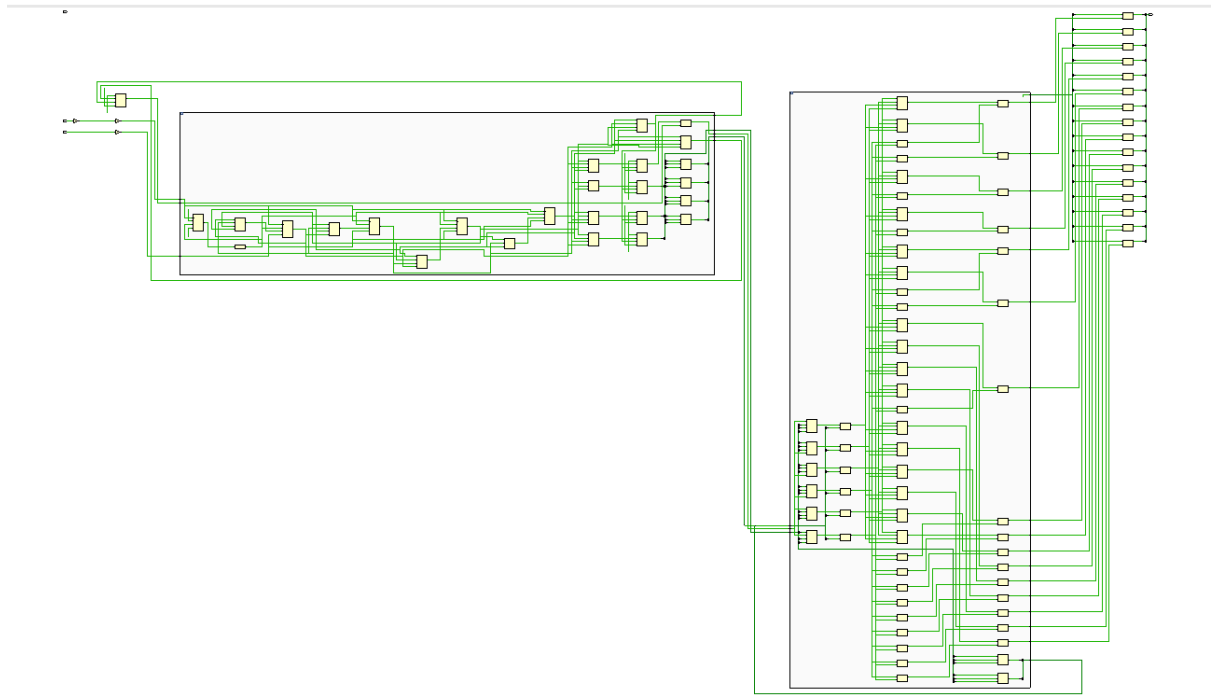


Device

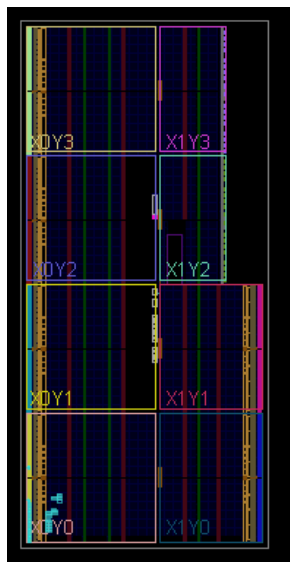


RTL schematic

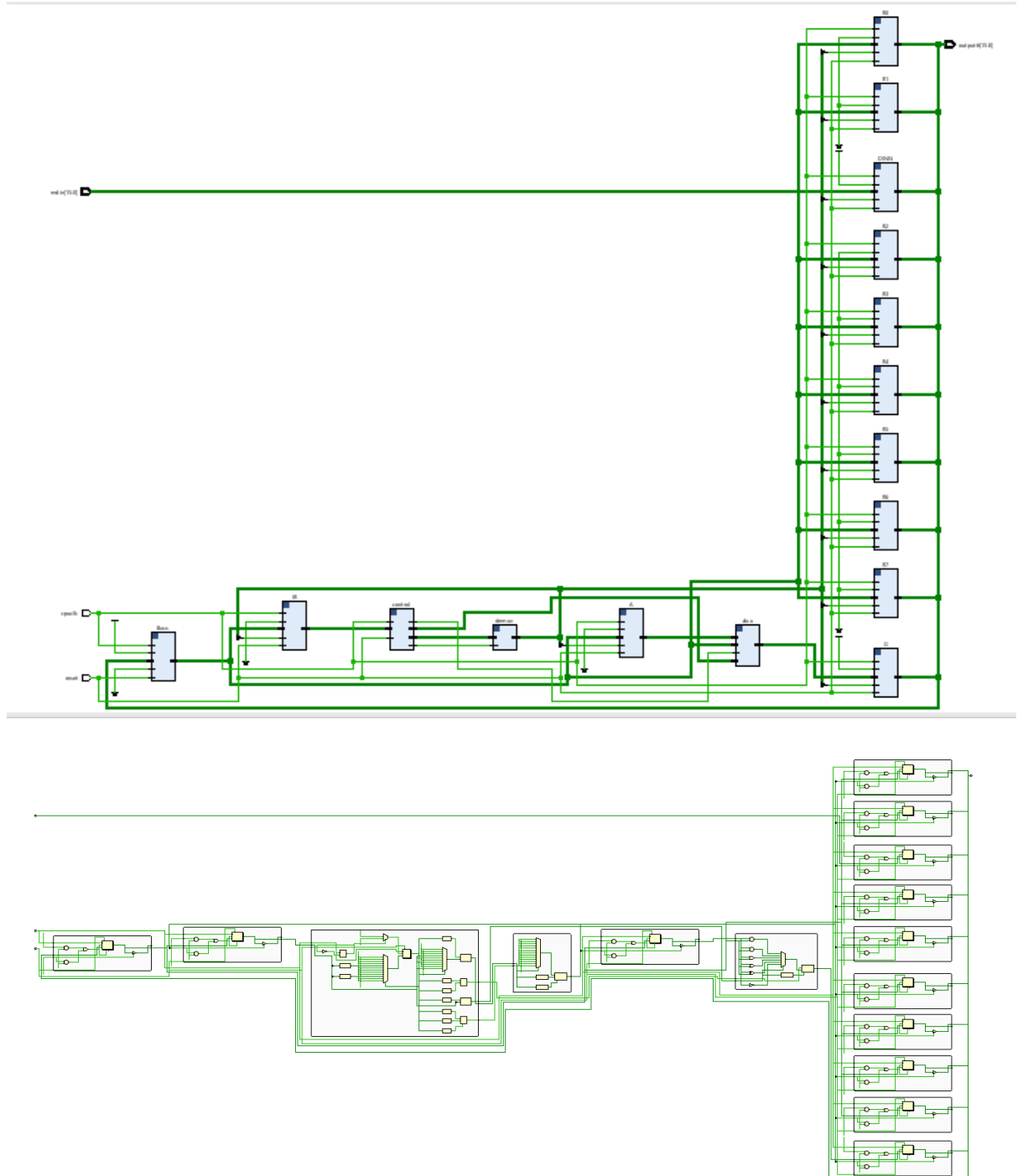




Device



RTL schematic of this structural code is



Note:

- For Simulations check the result waveforms in testbench, which contains working of all these components. Or open the project folder in to the submitted zip using VIVADO and run the simulation.
- The source code files can be found in “\Lab6_V2\Lab6_V2.srscs\sources_1\new” folder.

Inferences of Discussion

From the testbench results (inferences of clock cycles and correctness of processor)

- We can verify the correctness of each operation. The screenshot attached next to code is taken from the calculator. We can observe that our result matches.
- For the mv, mvi operations it took 8 clock cycles to complete. (As we used half frequency clock for FSM and normal frequency clock for registers. So, to complete 4 states in FSM it takes 8 clock cycles)
- For the rest of operation which require the ALU, it took 8 clock cycles of FSM i.e. 16 clock cycles to complete.

Conclusion

By this lab project we are able to understand the complete working of a simple processor and able to implement it in VHDL.

(And also

- Compared and verified our results of Operations
- Analysed the number of required clock cycles and triggering.)

References

[1]

<https://www.partitionwizard.com/partitionmanager/cpu-architecture.html>

The screenshots of Binary calculator are taken from

<https://www.rapidtables.com/calc/math/binary-calculator.html>

All the rest of the images are of my own simulations and drawings.

-Thank you-